

Name: Divyansh Bhardwaj

RA2211026030052

CSE AIML - A

Q1. One-Hot Representation vs Distributed Representation

Aspect	One-Hot Representation	Distributed Representation
Definition	Each word is represented by a sparse vector with one element as '1' and all others '0'.	Each word is represented by a dense, low-dimensional vector of real numbers.
Dimensionality	Equal to the vocabulary size (e.g., 10,000 words → 10,000-dim vector).	Usually much smaller (e.g., 100–300 dimensions).
Similarity	No inherent relationship between words; cosine similarity between any two words = 0.	Captures semantic similarity — similar words have similar vector values.
Learning	Manually assigned or index-based.	Learned automatically using neural models like Word2Vec, GloVe, etc.
Memory Efficiency	Inefficient due to high sparsity.	Efficient and compact.

Distributed representations are preferred in modern NLP models because they effectively capture both semantic and syntactic relationships between words, allowing models to understand meaning and context more deeply (for example, $king - man + woman \approx queen$ demonstrates how relationships are preserved in the vector space). Unlike one-hot representations, which treat every word as unrelated, distributed embeddings enable generalization to unseen contexts, helping models infer the meaning of new or rare words based on their proximity to known ones. Additionally, since distributed representations are dense and low-dimensional, they significantly reduce computational and memory requirements, making large-scale language modeling more efficient. These advantages make them an essential foundation for modern deep NLP architectures such as Transformers, BERT, and GPT, which rely on such embeddings to achieve state-of-the-art performance in understanding and generating human language.

Q2. Continuous Bag of Words (CBOW) and Skip-Gram (Word2Vec)

CBOW Model

- Predicts the **target word** from surrounding **context words**.
- Example: Given “The ____ is barking”, context = [The, is, barking], target = “dog”.
- Architecture:
 - Input: Context words’ embeddings (averaged/summed).
 - Hidden Layer: Shared weights (projection matrix).
 - Output: Softmax over vocabulary to predict target word.
- **Captures frequent co-occurrence** and smooths word meaning via shared contexts.

Skip-Gram Model

- Reverse of CBOW — predicts **context words** given a **target word**.
- Example: Given target “dog”, predicts likely contexts (“barks”, “pet”, “tail”).
- Works better with **rare words** and **large corpora**.

Both the CBOW and Skip-Gram models learn **word embeddings** in such a way that words appearing in **similar contexts** end up having **similar vector representations**. This enables the model to understand not just the occurrence of words but also their **semantic relationships**. As a result, operations on these vectors can reveal meaningful linguistic patterns — for example, the relationship $king - man + woman \approx queen$ demonstrates how **vector arithmetic** can capture analogies and conceptual similarities. This ability to encode meaning and relationships between words makes Word2Vec embeddings highly valuable for various **natural language processing tasks** such as sentiment analysis, translation, and text classification.

Q3. Architecture and Working of CNNs

Architecture Overview

A **Convolutional Neural Network (CNN)** processes input data (e.g., images) through:

1. **Convolution Layers:** Apply filters (kernels) to extract spatial features.
2. **Activation (ReLU):** Introduces non-linearity.
3. **Pooling Layers:** Downsample to reduce spatial size and computation.
4. **Fully Connected Layers:** Combine extracted features for classification or regression.

Key Components

Component	Function
-----------	----------

Kernel/Filter	Small matrix (e.g., 3×3) that slides over the image to detect features like edges, textures, etc.
Stride	Step size for moving the kernel; larger stride reduces output size.
Padding	Adds extra pixels around input to preserve spatial dimensions (e.g., ‘same’ vs ‘valid’).
Max Pooling	Takes the maximum value from a patch; helps in translation invariance and dimensionality reduction.

In a Convolutional Neural Network (CNN), several components play key roles in extracting and processing image features. The kernel or filter is a small matrix, typically of size 3×3 or 5×5, that slides over the image to detect important features such as edges, textures, and patterns. The stride determines how many pixels the kernel moves at each step; a larger stride results in a smaller output feature map, thereby reducing computational complexity. Padding involves adding extra pixels (usually zeros) around the input image to maintain its spatial dimensions and prevent information loss at the borders — commonly used modes include ‘*same*’ and ‘*valid*’ padding. Finally, max pooling selects the maximum value from a specific patch of the feature map, which helps achieve translation invariance and reduces dimensionality, retaining only the most significant features for further layers in the network.

Q4. AlexNet vs GoogLeNet vs ResNet

Model	Year	Key Innovation	Architecture Highlights	Impact
AlexNet	2012	Deep CNN with ReLU, Dropout, and GPU training	8 layers, 60M parameters	Started deep learning revolution; won ImageNet 2012
GoogLeNet (Inception)	2014	Inception modules combining multiple filter sizes	22 layers, fewer parameters due to 1×1 conv	Improved efficiency and multi-scale feature extraction
ResNet	2015	Introduced Residual Connections (skip connections)	152 layers possible without vanishing gradients	Enabled ultra-deep networks and became backbone of modern CNNs

The development of models like AlexNet, GoogLeNet, and ResNet had a profound influence on the evolution of deep convolutional neural networks. AlexNet marked a major breakthrough in 2012 by demonstrating the power of deep learning on large-scale image classification tasks,

using innovations such as ReLU activation, dropout regularization, and GPU-based training to achieve unprecedented accuracy on the ImageNet dataset. GoogLeNet (Inception) further advanced efficiency by introducing Inception modules, which combined multiple convolution filters of different sizes within the same layer, allowing the network to capture features at multiple scales while drastically reducing the number of parameters. ResNet revolutionized deep learning in 2015 with its residual (skip) connections, which solved the vanishing gradient problem and enabled the training of extremely deep networks with hundreds of layers.

Overall, these models paved the way for deeper, more efficient, and highly generalizable convolutional architectures, laying the foundation for modern computer vision systems used in tasks like image recognition, object detection, and scene understanding.

Q5. Recurrent Neural Networks (RNNs)

Definition

RNNs are neural networks designed for **sequential data** — where output depends on previous inputs (e.g., text, time-series).

Working

- At each time step t , RNN takes input x_t and previous hidden state $h_{t-1} \rightarrow$ computes new state h_t .
- Allows **memory of past information**.

Backpropagation Through Time (BPTT)

- Extends standard backpropagation across sequence steps.
- Gradients are propagated backward through many time steps.

Problems

- **Exploding Gradients:** Gradients grow exponentially $\rightarrow$ unstable training.
- **Vanishing Gradients:** Gradients shrink $\rightarrow$ network forgets long-term dependencies.

Backpropagation Through Time (BPTT) is an extension of the standard backpropagation algorithm used to train Recurrent Neural Networks (RNNs). It works by unrolling the RNN across several time steps and then propagating the gradients backward through each step, allowing the network to learn dependencies over sequences. However, this process can lead to two major issues. The exploding gradient problem occurs when gradients grow exponentially during backpropagation, causing unstable updates and divergence in training. Conversely, the

vanishing gradient problem arises when gradients become extremely small, making the network unable to learn long-term dependencies, as the effect of earlier inputs diminishes over time.

To address these challenges, techniques like gradient clipping are used to limit the magnitude of gradients, while advanced architectures such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks are designed to preserve and control information flow more effectively, thereby enabling stable and efficient learning of long-range dependencies.

Q6. Encoder–Decoder Model and Attention

Encoder–Decoder Model

- Used in **Sequence-to-Sequence (Seq2Seq)** tasks like translation or summarization.
- **Encoder:** Processes input sequence and compresses information into a **context vector**.
- **Decoder:** Generates output sequence token by token using that context.

Example:

Input: “Je t’aime” → Output: “I love you”.

Problem:

Fixed-length context vector struggles with **long sentences**.

Attention Mechanism:

- Allows the decoder to **focus on relevant parts** of the input sequence for each output step.
- Computes **weights (importance)** for each encoder state.

The attention mechanism offers significant benefits in sequence-to-sequence models by allowing the network to dynamically focus on the most relevant parts of the input while generating each output token. This selective focus not only improves overall model performance but also enhances alignment between input and output sequences—for instance, helping the model correctly match words in machine translation tasks. Additionally, attention provides an interpretable focus through *attention maps*, which visually show which input elements the model attends to at each step. This interpretability helps researchers and developers understand model behavior and trust its decisions more effectively.

Q7. Attention Mechanism and Its Types

Definition

Attention computes a **weighted sum of input features**, giving more importance to relevant elements for a specific task.

Formula

$$Attention(Q, K, V) = softmax \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Where Q = Query, K = Key, V = Value.

Types of Attention

Type	Description	Application
Spatial Attention	Focuses on specific regions of an image.	Image captioning, object detection.
Temporal Attention	Focuses on relevant time steps in sequences.	Speech recognition, translation.
Hierarchical Attention	Applies attention at multiple levels (e.g., words → sentences).	Document classification.
Self-Attention	Each word attends to all others in the same sequence.	Transformers (BERT, GPT).

The **attention mechanism** has a wide range of practical applications across different domains of artificial intelligence. In **image captioning**, attention allows the model to selectively focus on specific regions of an image while generating each word in the caption, ensuring that the description corresponds to the relevant visual details. For example, when describing “a dog playing with a ball,” the model can focus on the dog while generating “dog” and then shift attention to the ball while generating “ball.” In **language modeling**, attention helps the network efficiently capture **long-range dependencies** within text, enabling it to understand relationships between distant words in a sentence. This leads to more coherent, context-aware, and accurate text generation, which is crucial for tasks such as translation, summarization, and question answering.

Q8. Generative Adversarial Networks (GANs)

Definition

A GAN consists of two competing networks:

- **Generator (G):** Produces fake data (images, sounds, etc.).
- **Discriminator (D):** Distinguishes between real and fake data.

Architecture

1. **Generator:** Random noise → Neural network → Fake sample.
2. **Discriminator:** Receives both real and fake data → Outputs probability of being real.
3. **Training:**
 - G tries to **fool D** by improving fake samples.
 - D tries to **catch G** by improving classification accuracy.
 - Both train in a **minimax game**:

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p_z} [\log(1 - D(G(z)))]$$

Applications

1. **Image Synthesis / Deepfake Generation** — creating realistic human faces (e.g., StyleGAN).
2. **Image-to-Image Translation** — converting sketches to photos or day-to-night images (e.g., Pix2Pix, CycleGAN).
3. **Data Augmentation** — generating synthetic training data for ML models.

In summary, Generative Adversarial Networks (GANs) have transformed the field of computer vision and generative modeling by enabling the creative generation of highly realistic data. Through the adversarial training process between the generator and discriminator, GANs learn to produce outputs that closely resemble real-world data, such as lifelike images, videos, and even human faces. Their ability to synthesize realistic and diverse samples has revolutionized applications like image synthesis, style transfer, data augmentation, and deepfake creation, making GANs one of the most impactful innovations in modern artificial intelligence.